



Mediator Design Pattern

Prepared BY

Team StackNova

University of Barishal

Instructor

Md Samsuddoha

Assistant Professor
University of Barishal



Table of Contents

- 1 Introduction to Mediator Design Pattern
- 2 Intent of the Pattern
- 3 Why This Pattern Is Needed
- 4 Main Problem
- 5 Improved Solution
- 6 Key Features of Mediator Pattern
- 7 Structure of Mediator Pattern
- 8 Real-World Analogy
- 9 How Mediator Works Step by Step
- 10 Pros and Cons
- 11 When to Use Mediator Pattern
- 12 When Not to Use Mediator Pattern
- 13 Coding Example 1: Air Traffic Controller
- 14 Coding Example 2: Smart Home



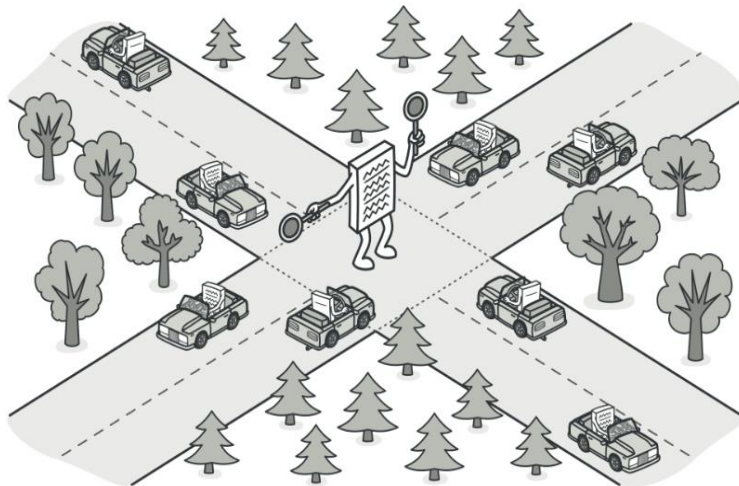
Introduction to Mediator Design Pattern

What is it?

- Mediator is a behavioral design pattern.
- It defines a separate object that handles communication between other objects.
- Objects do not communicate directly with each other.
- They send messages or events to the mediator instead.



Mediator = a middle controller that manages communication.





Intent of the Pattern

Intent: reduce direct dependencies between objects by making them communicate through a mediator object.



Less dependent — Objects no longer hold references to many other objects.



Centralized — All communication flows through one coordinating place.



Maintainable — Changing the interaction logic means changing the mediator.



Reusable — Components can be moved to other programs more easily.

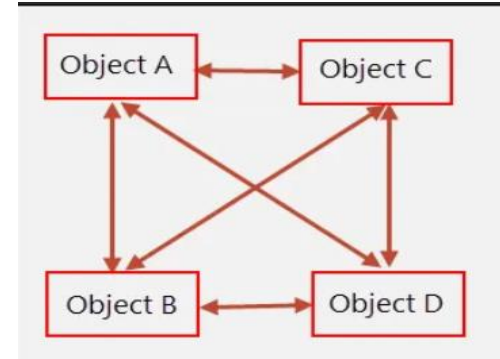
“Instead of many objects controlling each other, one mediator coordinates them.”



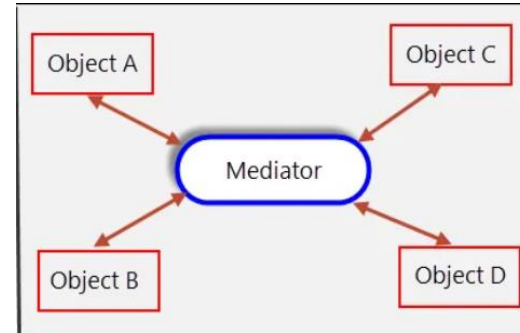
Why This Pattern Is Needed

The reality of real software

- Many objects often need to interact with each other.
- As the number of objects grows, the number of connections grows much faster.
- If every object knows about every other object, the system becomes tightly coupled.
- Tightly coupled code is hard to change, test, and reuse.



Without a mediator, every component may need to know about the others.





Problem

A flight booking system where every module talks to other modules directly

For say, a **Flight Booking System** that contains different modules such as **Passenger Form**, **Payment Module**, **Ticket Module**, **Notification Module**, and **Admin Panel**.

Some modules need to interact with others. For example:

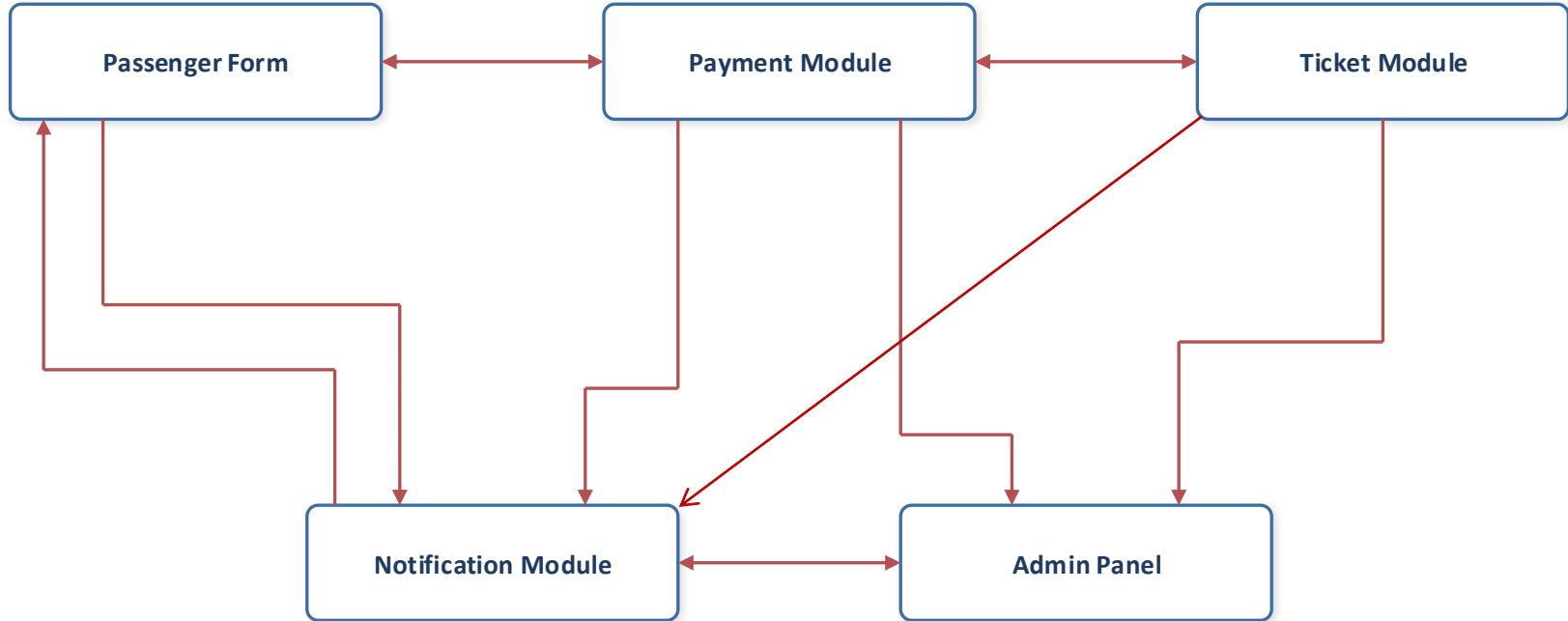
- After the passenger fills the **Passenger Form**, the information is sent to the **Payment Module**.
- After successful payment, the **Ticket Module** generates the ticket.
- Then the **Notification Module** sends booking confirmation messages.
- The **Admin Panel** receives updates to manage booking information.

If this communication logic is implemented directly inside each module, the modules become highly dependent on one another.



Before Mediator: Direct and Messy Communication

A flight booking system where every module talks to other modules directly



Every module knows about other modules — 8+ direct connections for only 5 modules.



Solution: Use a Mediator Object

The idea

- Introduce one central mediator object.
- All components talk to the mediator — never to each other.
- The mediator receives events and decides what should happen next.

What we gain

- Components become independent
- Communication controlled from one place
- Changes become easier
- Cleaner, easier-to-read structure

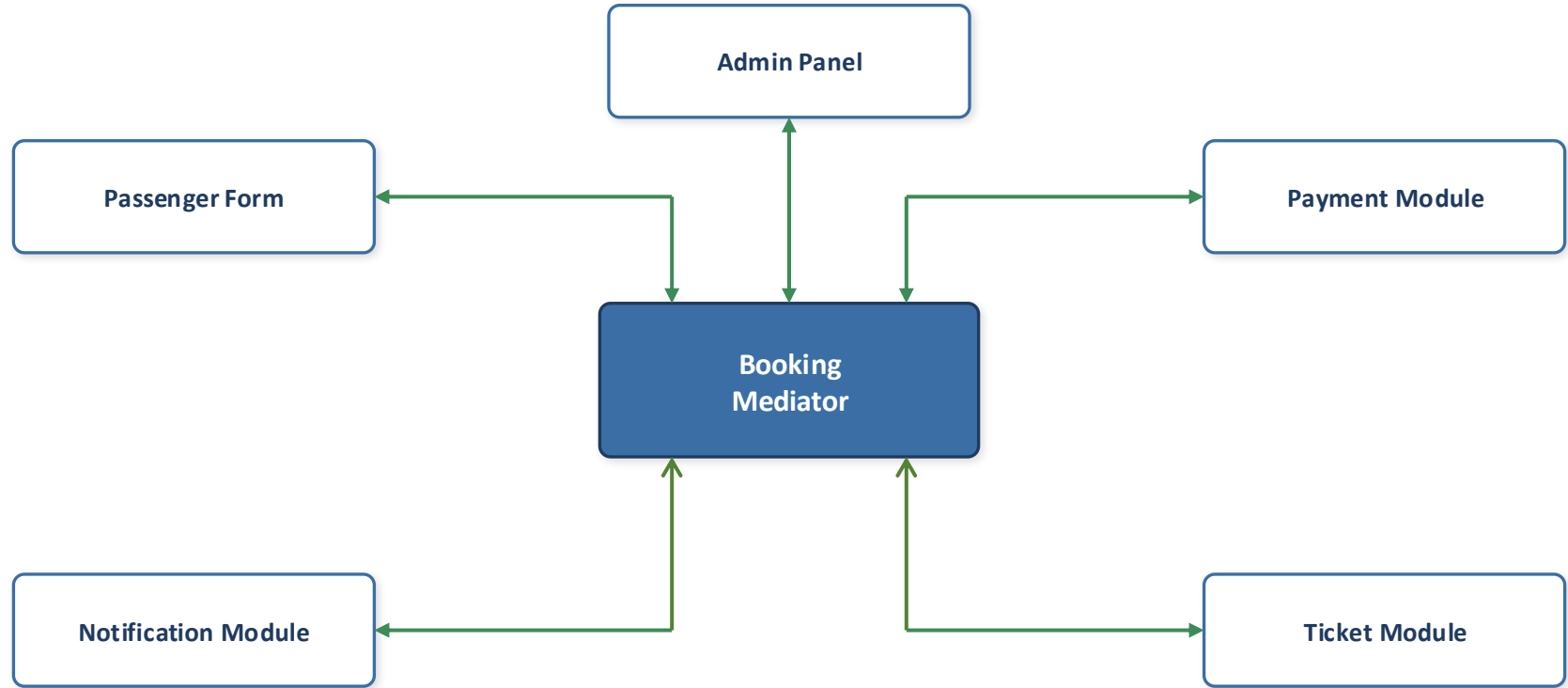
In the flight booking system:

Passenger Form sends a request to the Booking Mediator → the Mediator contacts Payment → creates the Ticket → sends the Notification → the Admin Panel gets status through the Mediator.



After Mediator: Centralized Communication

The same flight booking system — now every module connects only to the mediator



Only 5 connections — one per module. Adding a new module adds just one new connection.



Key Features of Mediator Pattern

1

Centralized Communication

All object interactions are handled by one mediator.

2

Loose Coupling

Components do not directly depend on each other.

3

Better Reusability

Components are free of references to many other classes.

4

Cleaner Object Interaction

The communication flow is easier to follow and understand.

5

Event-Based Coordination

Components notify the mediator about events: click, submit, select, change.

6

Improved Maintainability

Communication-logic changes happen mostly inside the mediator.



Structure of Mediator Pattern

1. Mediator Interface

Declares the communication method, for example notify(sender, event).

2. Concrete Mediator

Implements the interface and controls communication between components.

3. Component (base class)

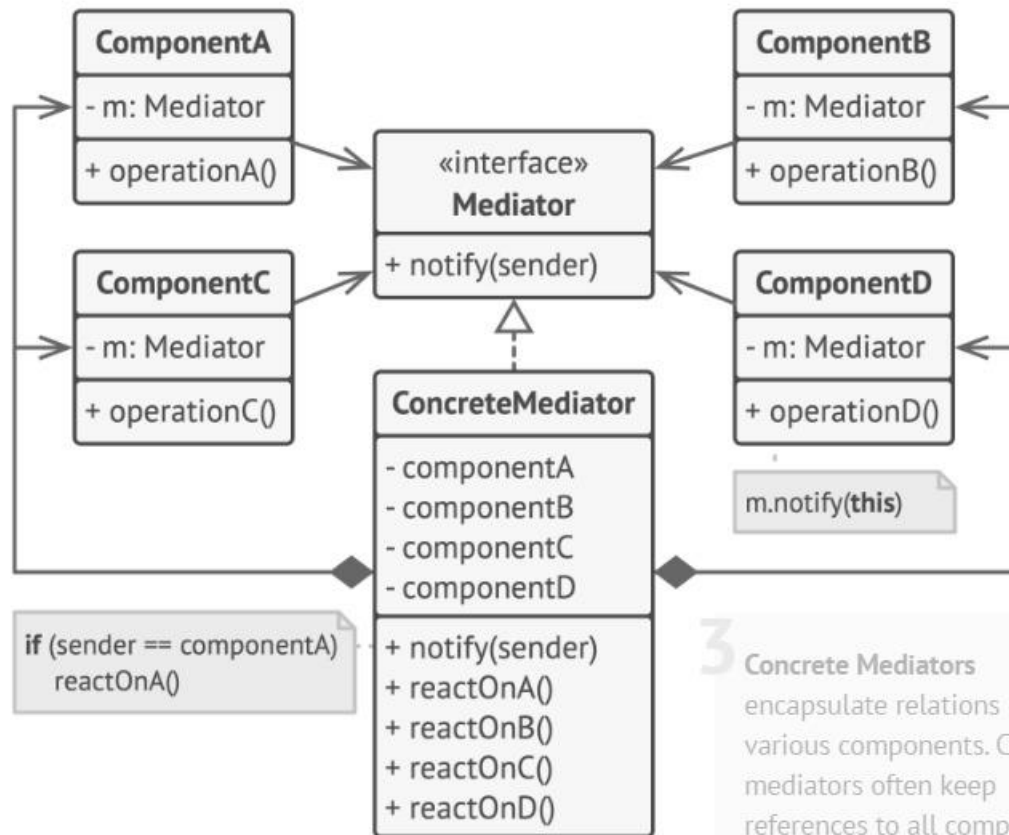
A general object that stores a reference to the mediator.

4. Concrete Components

The actual objects (buttons, sensors, users) that send events to the mediator.



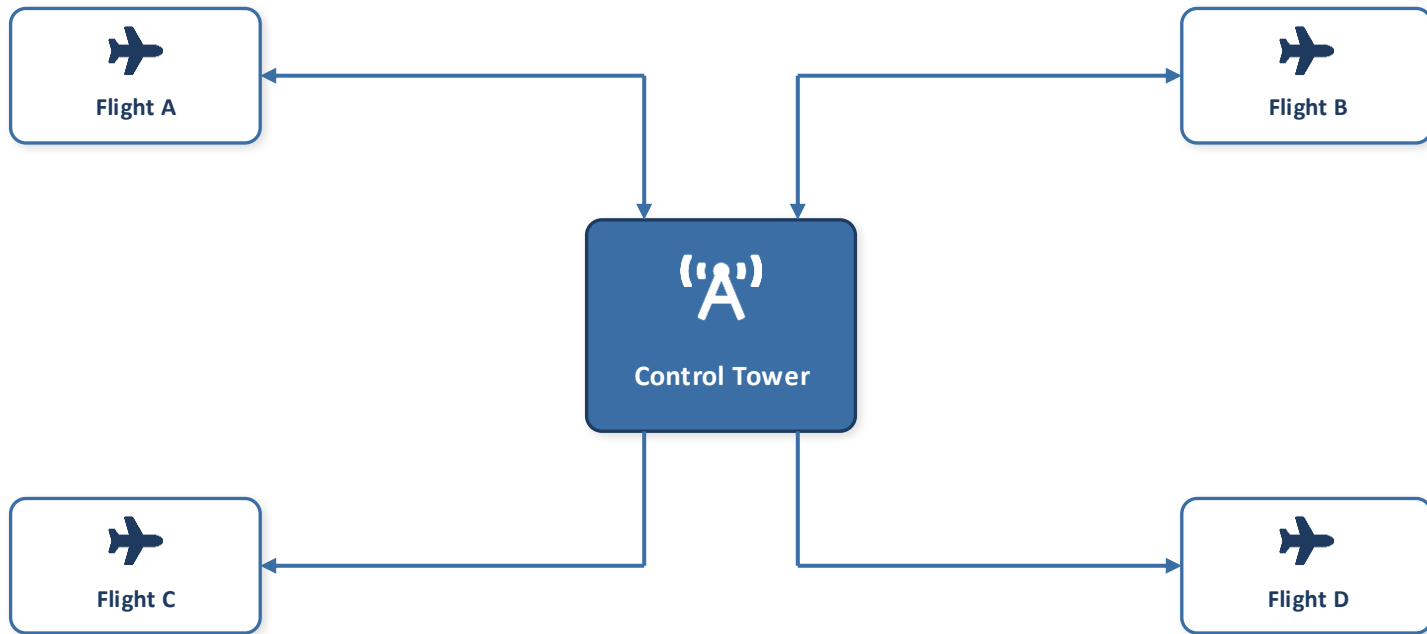
UML Structure Diagram



3 Concrete Mediators
encapsulate relations between various components. Concrete mediators often keep references to all components they manage and sometimes

'A'

Real-World Analogy: Air Traffic Control Tower



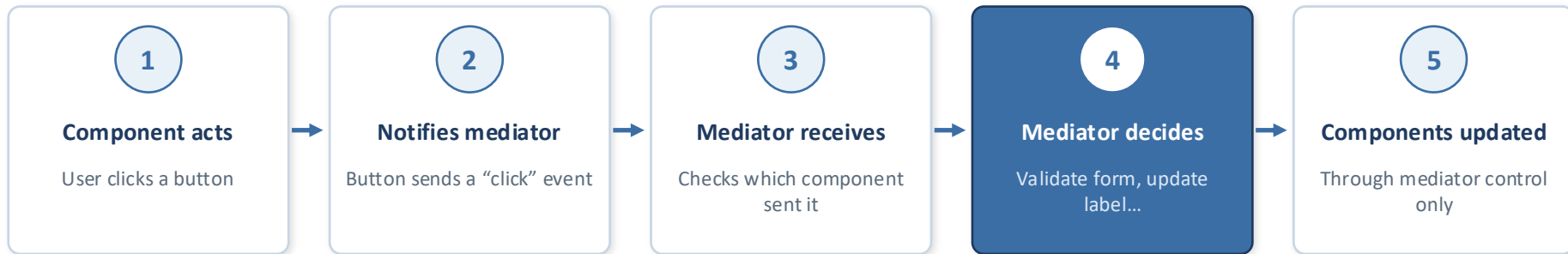
Airplanes = Components

Control Tower = Mediator

Landing / takeoff instructions = Events



How Mediator Works Step by Step



The component never needs to know the full process — it only sends its event to the mediator.

Component Action → Notify Mediator → Mediator Checks Event → Mediator Decides → Components Updated



Pros and Cons



Pros

- Reduces direct dependencies between objects
- Makes code easier to maintain
- Improves reusability of components
- Keeps communication logic in one place
- Makes complex interaction easier to manage
- Supports cleaner overall design



Cons

- Mediator class can become too large
- Too much logic creates a “God Object”
- Debugging is harder when many events pass through one place
- Not useful for very simple systems



When to Use Mediator Pattern

Use it when...

- Many objects communicate with each other
- Object relationships are becoming complex
- You want to reduce tight coupling
- You want reusable components
- Communication logic is scattered across many classes
- UI components need organized event handling
- Workflow or process control is needed

Typical examples

- Chat application
- Login form
- Booking system
- Air traffic control system
- Order processing system
- Smart home control system



When Not to Use Mediator Pattern



Very small system

A handful of objects with obvious interactions does not need a coordinator.



Simple relationships

If A only ever talks to B, a mediator just adds a detour.



Already clear communication

Don't fix what is already easy to read and maintain.



Mediator adds complexity

If the pattern makes the code longer and harder, it is the wrong tool.



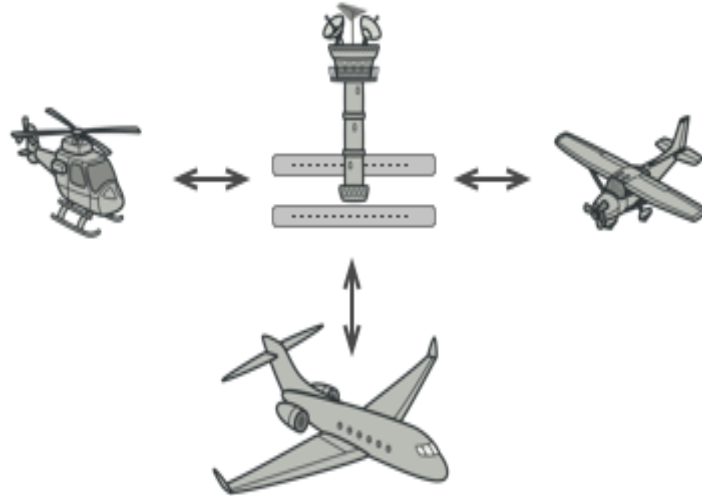
Pure message-passing

A mediator that only forwards calls without logic adds no value.

Design patterns should solve a real problem. No communication problem → no mediator needed.



Example 1: Air Traffic Control Tower— Scenario



Airplanes = Components

Control Tower = Mediator

Landing / takeoff instructions = Events

File Name : Main.java

```
public class Main {  
  
    public static void main(String[] args) {  
  
        // create ATC (middle controller)  
        ATC atc = new ATC();  
  
        // create plane A  
        Airplane planeA = new Airplane("Plane A", atc);  
  
        // create plane B  
        Airplane planeB = new Airplane("Plane B", atc);  
  
        // plane A asks to land  
        planeA.requestLanding();  
  
        // plane B asks to takeoff  
        planeB.requestTakeoff();  
  
        // plane A finishes landing  
        planeA.finish();  
  
        // plane B tries again  
        planeB.requestTakeoff();  
    }  
}
```

File name : Airplane.java

```
class Airplane {  
  
    String name; // airplane name  
    ATC atc;     // connect to ATC  
  
    // set name and ATC  
    Airplane(String name, ATC atc) {  
        this.name = name;  
        this.atc = atc;  
    }  
  
    // ask for landing  
    void requestLanding() {  
  
        System.out.println(name + " wants to LAND");  
  
        atc.landingRequest(name); // send request to ATC  
    }  
}
```

```
// ask for takeoff
void requestTakeoff() {

    System.out.println(name + " wants to TAKE OFF");

    atc.takeoffRequest(name); // send request to ATC
}

// tell ATC runway is free
void finish() {

    atc.runwayFree();
}
}
```

File name : ATC.java (AirTrafficControl)

```
class ATC {  
  
    // runway is free or not  
    private boolean runwayFree = true;  
  
    // airplane asks for landing  
    public void landingRequest(String planeName) {  
  
        // check runway is free  
        if (runwayFree) {  
            runwayFree = false; // now runway is busy  
  
            // allow landing  
            System.out.println("ATC: " + planeName + " can LAND now.");  
        } else {  
            // runway is busy  
            System.out.println("ATC: Runway busy. " + planeName + " wait.");  
        }  
    }  
}
```

```
// airplane asks for takeoff
public void takeoffRequest(String planeName) {

    // check runway is free
    if (runwayFree) {
        runwayFree = false; // runway is now used

        // allow takeoff
        System.out.println("ATC: " + planeName + " can TAKE OFF now.");
    } else {
        // runway is busy
        System.out.println("ATC: Runway busy. " + planeName + " cannot take off.");
    }
}

// runway becomes free again
public void runwayFree() {

    runwayFree = true; // free the runway

    System.out.println("ATC: Runway is now free.");
}
}
```

OUTPUT

Plane A wants to LAND

ATC: Plane A can LAND now.

Plane B wants to TAKE OFF

ATC: Runway busy. Plane B cannot take off.

ATC: Runway is now free.

Plane B wants to TAKE OFF

ATC: Plane B can TAKE OFF now.



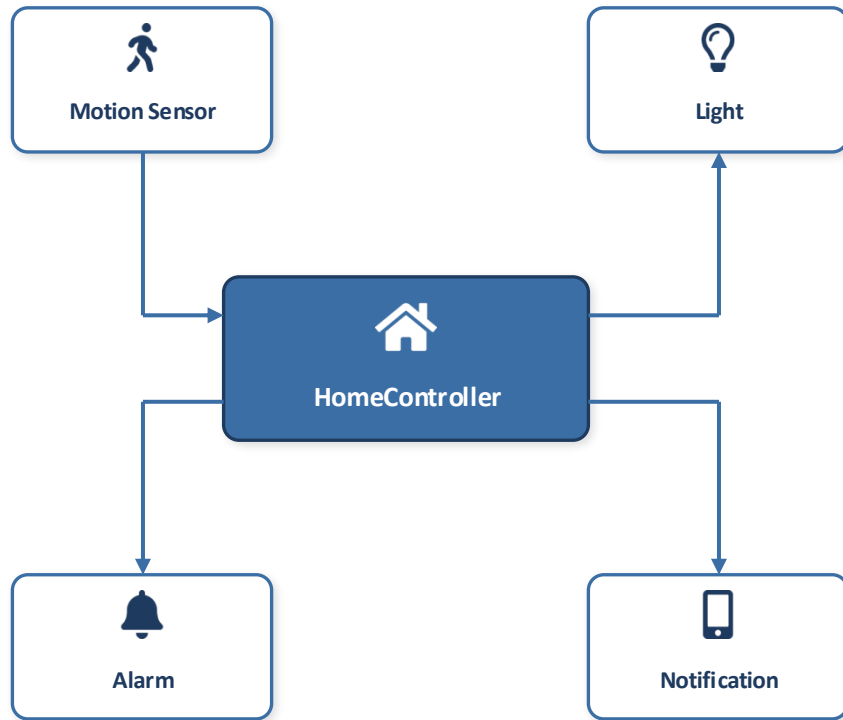
Example 2: Smart Home Mediator — Scenario

Scenario

- Smart home devices need to work together.
- When motion is detected: light turns on, alarm starts, owner is notified.
- Without a mediator, each device would need to know about the others.

Motion Sensor / Light / Alarm = Components

HomeController = Mediator



File Name : Main.java

```
public class Main {  
  
    public static void main(String[] args) {  
  
        // Create devices  
        Light light = new Light();  
        Alarm alarm = new Alarm();  
        Notification notification = new Notification();  
  
        // Create Mediator  
        HomeController controller = new HomeController(light, alarm, notification);  
  
        // Create Motion Sensor  
        MotionSensor sensor = new MotionSensor("Motion Sensor", controller);  
  
        // Detect motion  
        sensor.detectMotion();  
    }  
}
```

File Name : HomeController.java

```
class HomeController {

    Light light;
    Alarm alarm;
    Notification notification;

    HomeController(Light light, Alarm alarm, Notification notification) {
        this.light = light;
        this.alarm = alarm;
        this.notification = notification;
    }

    public void motionDetected(String sensorName) {

        System.out.println(sensorName + " detected motion.");

        light.turnOn();
        alarm.start();
        notification.send();
    }
}
```

File Name : MotionSensor.java

```
class MotionSensor {  
  
    String name;  
    HomeController controller;  
  
    MotionSensor(String name, HomeController controller) {  
        this.name = name;  
        this.controller = controller;  
    }  
  
    void detectMotion() {  
        System.out.println(name + " detects motion.");  
        controller.motionDetected(name);  
    }  
}
```

File Name : Light.java

```
class Light {  
  
    void turnOn() {  
        System.out.println("Light is ON.");  
    }  
}
```

File Name : Alarm.java

```
class Alarm {  
  
    void start() {  
        System.out.println("Alarm is STARTED.");  
    }  
}
```

File Name : Notification.java

```
class Notification {  
  
    void send() {  
        System.out.println("Owner is NOTIFIED.");  
    }  
}
```

OUTPUT

Motion Sensor detects motion.

Motion Sensor detected motion.

Light is ON.

Alarm is STARTED.

Owner is NOTIFIED.



References

Refactoring Guru — Mediator

refactoring.guru/design-patterns/mediator (main reference: concept, structure, pros & cons)

Gamma, Helm, Johnson, Vlissides

Design Patterns: Elements of Reusable Object-Oriented Software (GoF, 1994) — original definition

Video Walkthrough — Mediator Pattern

youtu.be/9_bZ250Lc6o (supplementary explanation & live coding demo)



Thank You

Any Questions?

"Mediator makes object communication simple, clean, and organized."